

Сетевой уровень

11 июня 2026 г.

Сетевой уровень отвечает за передачу пакетов от отправителя получателю. Чтобы достичь пункта назначения, пакет может совершить множество скачков между маршрутизаторами.

Это резко контрастирует с деятельностью канального уровня, цель которого намного скромнее — просто перемещать фреймы с одного конца «провода» (виртуального) на другой.

Таким образом, сетевой уровень является самым нижним уровнем, имеющим дело с передачей данных по всему пути от начала до конца.

Для достижения этих целей сетевой уровень должен знать топологию сети (то есть весь набор маршрутизаторов и каналов) и рассчитывать подходящие маршруты, даже если она достаточно крупная.

При выборе пути он также должен заботиться о равномерной нагрузке на маршрутизаторы и линии связи.

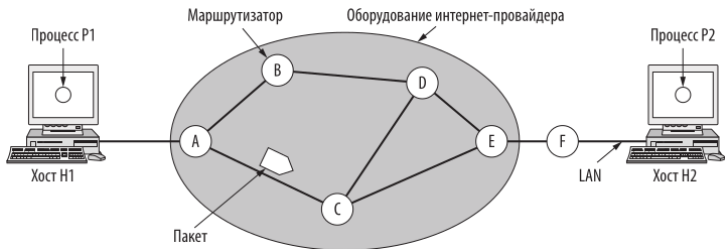
Наконец, когда источник и адресат находятся в разных независимо управляемых сетях (иногда называемых автономными системами), приходится решать ряд дополнительных вопросов, включая проблему координации потоков трафика в рамках нескольких сетей и управление загруженностью сети.

Эти проблемы обычно решаются на сетевом уровне; операторам сетей часто приходится делать это вручную.

Обычно они самостоятельно перенастраивали сетевой уровень путем изменения низкоуровневой конфигурации.

Однако с появлением программно-конфигурируемых сетей и программируемого оборудования появилась возможность настраивать сетевой уровень с помощью более эффективного ПО, и даже полностью переопределять его функции.

Метод коммутации пакетов с ожиданием



Метод коммутации пакетов с ожиданием

Система работает следующим образом. Хост, у которого есть пакет для передачи, отправляет его на ближайший маршрутизатор — либо в своей LAN, либо провайдеру по двухточечному соединению (например, по каналу ADSL или по линии кабельного телевидения).

Метод коммутации пакетов с ожиданием

Там пакет хранится до тех пор, пока не будет принят целиком и не пройдет полную обработку, включая верификацию контрольной суммы. Затем он передается по цепочке маршрутизаторов, которая в итоге приводит к пункту назначения. Такой механизм называется коммутацией пакетов с промежуточным хранением (store-and-forward), и мы уже рассматривали его в предыдущих главах.

Службы, предоставляемые транспортному уровню

Службы для транспортного уровня предоставляются сетевым уровнем посредством интерфейса между ними. Главное — определить, какими должны быть эти службы.

- ❶ Службы сетевого уровня не должны зависеть от технологии маршрутизатора.
- ❷ Количество, тип и топология имеющихся маршрутизаторов не должны влиять на транспортный уровень.
- ❸ Сетевые адреса, доступные транспортному уровню, должны использовать единую систему нумерации, даже между LAN и WAN.

С учетом этих условий разработчики абсолютно свободны в написании детальной спецификации служб для транспортного уровня. Эта свобода часто перерастает в яростную борьбу между двумя непримиримыми сторонами.

В центре дискуссии — вопрос о том, какие службы должен предоставлять сетевой уровень: с установлением соединения или без него.

Один лагерь (представленный интернет-сообществом) заявляет, что работа маршрутизаторов сводится к перемещению пакетов. С этой точки зрения (основанной на сорокалетнем опыте работы с реальными компьютерными сетями) сеть по своей природе ненадежна, независимо от того, как она спроектирована. Хосты должны учитывать это и самостоятельно защищаться от ошибок (то есть заниматься их обнаружением и коррекцией) и управлять потоком.

Из этого следует, что сетевой службе не нужно требовать установления соединения, она должна в основном состоять из примитивов SEND PACKET (отправить пакет) и RECEIVE PACKET (получить пакет).

В частности, сюда нельзя включать упорядочивание пакетов и контроль потока — все равно эти действия выполнит хост. Нет смысла выполнять одну и ту же работу дважды. Такое рассуждение — пример применения сквозного принципа (end-to-end argument), оказавшего значительное влияние на формирование интернета (Зальцер и др.; Saltzer et al., 1984). Кроме того, каждый пакет должен содержать полный адрес получателя, так как передача производится независимо от предшествующих пакетов.

Другой лагерь, представленный телефонными компаниями, утверждает, что сеть должна предоставлять надежную службу с установлением соединения. Компании считают, что 100 лет успешного управления телефонными системами по всему миру — серьезный аргумент в их пользу.

По их мнению, определяющим фактором является QoS, и без установления соединения в сети очень сложно добиться каких-либо приемлемых результатов, особенно когда дело касается трафика в реальном времени (например, передачи голоса и видео).

Мы рассмотрели два класса служб, которые сетевой уровень может предоставлять своим пользователям. Теперь перейдем к обсуждению его устройства. Возможны два варианта в зависимости от типа службы.

Если предоставляется служба без установления соединения, пакеты по отдельности передаются в сеть и их маршруты рассчитываются независимо. При этом никакой предварительной настройки не требуется. В этом случае пакеты часто называют **дейтаграммами** (datagrams), по аналогии с телеграммами, а сети — **дейтаграммными** (datagram network).

При использовании службы, ориентированной на установление соединения, весь путь от маршрутизатора-отправителя до маршрутизатора-получателя должен быть определен до начала передачи. Такое соединение называется **виртуальным каналом** (VC, Virtual Circuit), по аналогии с физическими каналами, устанавливаемыми в традиционных телефонных сетях. Сама система называется **сетью виртуального канала** (virtual-circuit network).

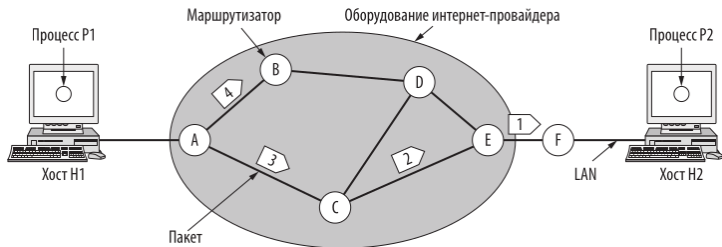


Таблица маршрутизатора A (в начале)

A	—
B	B
C	C
D	B
E	C
F	C

Назна-
чение

Линия

Таблица маршрутизатора A (в конце)

A	—
B	B
C	C
D	B
E	B
F	B

Таблица маршрутизатора C

A	A
B	A
C	—
D	E
E	E
F	E

Таблица маршрутизатора E

A	C
B	D
C	C
D	D
E	—
F	F

Рассмотрим принцип работы дейтаграммных сетей. Пусть у процесса P1 есть длинное сообщение для P2. Он передает свое послание транспортному уровню и информирует его о том, что данные необходимо доставить процессу P2 на хосте H2. Код транспортного уровня выполняется на хосте H1; более того, обычно он является частью операционной системы. Заголовок транспортного уровня вставляется в начало сообщения, и в таком виде оно передается на сетевой уровень. Как правило, это просто еще одна процедура операционной системы.

Предположим, что в нашем примере сообщение в четыре раза длиннее максимального размера пакета. Поэтому сетевой уровень должен разбить его на четыре части (1, 2, 3 и 4) и отправить их все поочередно на маршрутизатор А с использованием какого-нибудь протокола двухточечного соединения, например PPP. Здесь в игру вступает провайдер.

Каждый маршрутизатор имеет свою внутреннюю таблицу, по которой он определяет дальнейший путь пакета до любого возможного пункта назначения.

Каждая запись таблицы состоит из двух полей: адресат и ведущая к нему исходящая линия. Во втором поле могут использоваться только линии, непосредственно соединенные с данным маршрутизатором.

Так, например, на илл. у маршрутизатора А имеются только две исходящие линии: к В и к С. Поэтому все входящие пакеты передаются в какую-то из этих двух точек, даже если они не являются адресатами. Изначальная таблица маршрутизации А показана на илл. с пометкой «в начале».

В маршрутизаторе А пакеты 1, 2 и 3, поступившие на вход, кратковременно сохраняются для верификации контрольной суммы. Затем в соответствии с таблицей А каждый пакет отправляется в новом фрейме по исходящему соединению на маршрутизатор С. Далее пакет 1 уходит на Е, затем — на F. После этого он передается внутри фрейма по LAN на хост H2. Пакеты 2 и 3 следуют по тому же маршруту.

Однако с пакетом 4 происходит нечто другое. После прибытия на А он пересылается на маршрутизатор В, несмотря на то что его адресат — F, как и у первых трех пакетов. По какой-то причине А решил отправить пакет 4 по новому пути. Может быть, в результате отправки трех пакетов на линии АСЕ возник затор и маршрутизатор решил обновить свою таблицу (она показана на илл. с пометкой «в конце»).

Алгоритмы, управляющие таблицами и принимающие решения о выборе маршрута, называются **алгоритмами маршрутизации** (routing algorithms)

Протокол IP, составляющий основу всего интернета, является наиболее ярким примером сетевой службы без установления соединения. Каждый пакет содержит IP-адрес назначения, по которому маршрутизатор осуществляет индивидуальную отправку пакета. В пакетах IPv4 используются адреса длиной 32 бита, а в IPv6 — 128 бит.

Реализация службы с установлением соединения

Службе, ориентированной на установление соединения, нужна сеть виртуального канала.

Идея виртуальных каналов состоит том, чтобы не выбирать новый маршрут для каждого пакета, как на илл. Вместо этого путь от источника до адресата выбирается в процессе установления соединения и хранится в специальных таблицах, встроенных в маршрутизаторы.

Один и тот же маршрут используется для всего трафика, проходящего через данное соединение. Именно так работает телефонная система. Когда соединение разрывается, виртуальный канал прекращает свое существование. При использовании службы с установлением соединения каждый пакет включает в себя идентификатор виртуального канала.

Хост Н1 установил соединение с хостом Н2. Это соединение запоминается и становится первой записью во всех таблицах маршрутизации. Так, первая строка таблицы маршрутизатора А говорит о том, что если пакет с идентификатором соединения 1 пришел с хоста Н1, то его нужно направить на С с идентификатором соединения 1. Точно так же первая запись С направляет пакет на Е все с тем же идентификатором соединения 1.

Теперь выясним, что произойдет, если хост H3 захочет установить соединение с H2. Он выбирает идентификатор соединения 1 (на данный момент это первое и единственное соединение) и просит сеть установить виртуальный канал.

Таким образом, в таблице появляется вторая запись. Обратите внимание, что здесь возникает конфликт: А может отличить пакеты соединения 1, пришедшие с Н1, от пакетов соединения 1, пришедших с Н3, но С такой возможности не имеет. Поэтому А присваивает новый идентификатор соединения исходящему трафику и тем самым создает второе соединение. Для предотвращения таких конфликтов маршрутизаторы получили возможность менять идентификаторы соединения в исходящих пакетах.

Одним из примеров сетевой службы, ориентированной на соединение, является мультипротокольная коммутация по меткам (MultiProtocol Label Switching, MPLS). Она используется в сетях интернет-провайдеров; при этом IP-пакеты получают MPLS-заголовок, содержащий 20-битный идентификатор соединения или метку. Если провайдер устанавливает длительное соединение для передачи крупных объемов данных, MPLS часто остается невидимым для клиентов.

Проблема	Дейтаграммы	Виртуальные каналы
Установка канала	Не требуется	Требуется
Адресация	Каждый пакет содержит полный адрес отправителя и получателя	Каждый пакет содержит короткий номер виртуального канала
Информация о состоянии	Маршрутизаторы не содержат информации о состоянии	Каждый виртуальный канал требует места в таблице маршрутизатора
Маршрутизация	Маршрут каждого пакета выбирается независимо	Маршрут выбирается при установке виртуального канала. Каждый пакет следует по этому маршруту
Эффект от выхода маршрутизатора из строя	Никакого, кроме потерянных пакетов	Все виртуальные каналы, проходившие через отказавший маршрутизатор, прекращают существование
QoS	Трудно реализовать	Легко реализуется при наличии достаточного количества ресурсов для каждого виртуального канала
Борьба с перегрузкой	Трудно реализовать	Легко реализуется при наличии достаточного количества ресурсов для каждого виртуального канала

Виртуальный канал требует определенных временных затрат на установку, однако в результате это существенно упрощает обработку пакетов данных. Чтобы понять, куда отправить пакет, маршрутизатору требуется всего лишь обратиться к таблице, используя номер канала. Дейтаграммная сеть не требует установки, но адрес получателя определяется с помощью более сложной процедуры поиска.

В связи с этим возникает другая проблема: адреса назначения в дейтаграммах сетях — глобальные, поэтому они намного длиннее, чем номера линий в сетях виртуальных каналов. При сравнительно небольшом размере пакетов включение полного адреса в каждый из них может привести к существенным издержкам, а следовательно, к снижению пропускной способности.

Виртуальные каналы обладают некоторыми преимуществами в обеспечении QoS и предотвращении заторов в сети, так как ресурсы (например, буфер, пропускная способность, центральный процессор) можно зарезервировать заранее, во время установки соединения. Как только пакеты начнут приходить, необходимая пропускная способность и мощность маршрутизатора будут предоставлены. В дейтаграммной сети предотвращение заторов реализовать значительно сложнее.

В системах обработки транзакций (например, при запросе магазина на подтверждение оплаты картой) накладные расходы на установку соединения и удаление виртуального канала могут сильно снизить потребительские свойства сети. Если большая часть трафика будет такой, то использование виртуального канала не имеет особого смысла.

Однако в случае длительных операций, таких как обмен данными через VPN внутри одной компании, постоянные виртуальные каналы (установленные вручную на месяцы и даже годы) могут оказаться полезными.

Недостаток виртуальных каналов — уязвимость в случае выхода из строя или временного выключения маршрутизатора. Даже если его включат через пару секунд, все проходившие через него каналы прервутся.

Если же это произойдет в дейтаграммной сети, потеряются только те пакеты, которые находились на маршрутизаторе в данный момент (а скорее всего, даже они не пострадают, поскольку отправитель сразу же повторит передачу).

Обрыв линии связи для виртуальных каналов является фатальным, но легко компенсируется в дейтаграммной системе. Кроме того, в ней маршрутизаторы могут обеспечить баланс трафика по всей сети, изменяя путь в процессе долгой передачи.

Основная функция сетевого уровня заключается в маршрутизации пакетов от начальной до конечной точки. В этом разделе мы поговорим о том, как эта задача выполняется в рамках одного административного домена или одной автономной сети.

В большинстве сетей пакетам приходится проходить через несколько маршрутизаторов. Единственное исключение — широковещательные сети, но даже в них маршрутизация представляет собой проблему, если отправитель и получатель находятся в разных сегментах сети. Алгоритмы, выбирающие путь, и используемые ими структуры данных составляют основу проектирования сетевого уровня.

Алгоритм маршрутизации (routing algorithm) — это компонент программного обеспечения сетевого уровня, отвечающий за выбор выходной линии для отправки поступившего пакета.

Если сеть использует дейтаграммную службу, выбор маршрута должен производиться заново для каждого пакета, так как оптимальный путь мог измениться.

При использовании виртуальных каналов маршрут определяется только при создании нового канала, и после этого по нему следуют все пакеты данных. Этот подход называют сеансовой маршрутизацией (session routing), так как путь существует на протяжении всего сеанса связи (например, пока вы подключены к VPN).

Важно различать маршрутизацию, при которой система выбирает путь для пакета, и пересылку, происходящую при его получении. Можно представить себе маршрутизатор как устройство, в котором выполняются два процесса.

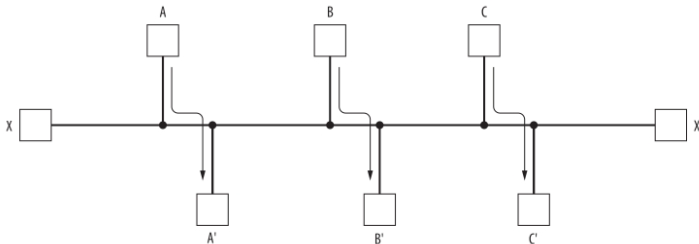
Один обрабатывает приходящие пакеты и выбирает для них исходящую линию по таблице маршрутизации; он называется **пересылкой** (forwarding).

Второй процесс отвечает за заполнение и обновление таблиц. Именно для этого нужен алгоритм маршрутизации.

Вне зависимости от того, как выбирается путь — отдельно для каждого отправляемого пакета или только один раз при установлении нового соединения, — желательно, чтобы алгоритм маршрутизации обладал определенными свойствами: корректностью, простотой, надежностью, устойчивостью, равнодоступностью и эффективностью.

Хосты, маршрутизаторы и линии связи будут постоянно выходить из строя, а топология неоднократно изменится. Алгоритм маршрутизации должен уметь с этим справляться, не прерывая выполнение задач на всех хостах. Представьте себе сеть, которая перезагружается при каждой поломке маршрутизатора!

Алгоритм маршрутизации также должен быть устойчивым. Существуют алгоритмы, которые никогда не сводятся к созданию фиксированного набора путей, независимо от того, как долго они работают. Устойчивый алгоритм достигает равновесия и остается в этом состоянии. Но он также должен быстро находить более подходящий набор маршрутов, поскольку соединение может прерваться до того, как будет достигнуто равновесие.



Такие свойства, как равнодоступность и эффективность, могут показаться очевидными — вряд ли кто-нибудь станет возражать против них, — однако они зачастую несовместимы. Для примера рассмотрим ситуацию на илл. Предположим, что трафик между станциями A и A', B и B', а также C и C' настолько интенсивный, что горизонтальные линии связи полностью загружены. Чтобы

Прежде чем начинать поиск приемлемого соотношения равнодоступности и эффективности, следует решить, что именно нужно оптимизировать. Для увеличения эффективности передачи данных по сети можно минимизировать среднее время задержки или увеличить общую пропускную способность.

Однако эти действия также противоречат друг другу, поскольку работа любой системы с очередями ожидающих обработки пакетов, да еще и на максимуме производительности оборудования, приводит к увеличению времени ожидания.

В качестве компромисса многие сети пытаются минимизировать расстояние, которое должен пройти пакет, или снизить количество пересылок для каждого пакета. В обоих случаях уменьшается задержка и объем полосы, требуемый для каждого пакета, благодаря чему повышается пропускная способность всей сети.

Алгоритмы маршрутизации можно разделить на два основных класса: неадаптивные и адаптивные.

Неадаптивные алгоритмы (nonadaptive algorithms) не учитывают текущую топологию и не измеряют трафик. Вместо этого путь для каждой пары станций определяется заранее, в автономном режиме, а список маршрутов загружается в маршрутизаторы во время загрузки сети. Эта процедура называется **статической маршрутизацией** (static routing).

Она не реагирует на сбои, поэтому обычно используется в тех случаях, когда выбор маршрута очевиден. Например, маршрутизатор F на илл. 5.3 должен отправлять пакеты в сеть, на маршрутизатор E, независимо от конечного адреса назначения.

Адаптивные алгоритмы (adaptive algorithms), напротив, реагируют на изменения топологии, а иногда и на загруженность линий. Такие алгоритмы **динамической маршрутизации** (dynamic routing) различаются по нескольким параметрам.

Они могут получать информацию от соседних маршрутизаторов или от всех маршрутизаторов сети. Некоторые меняют путь при смене топологии, другие — через определенные равные интервалы времени по мере изменения нагрузки. Наконец, для оптимизации они используют разные показатели: расстояние, количество транзитных участков (переходов) или ожидаемое время передачи.

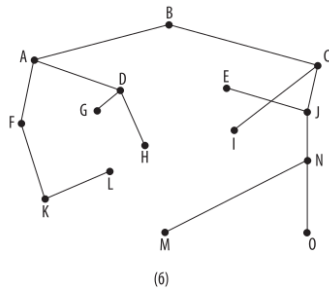
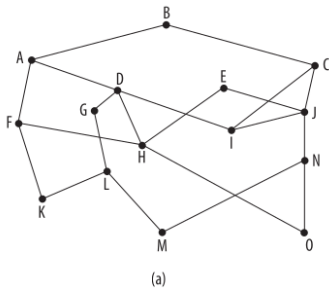
В следующих разделах мы рассмотрим множество алгоритмов маршрутизации. Помимо передачи пакета от источника к месту назначения, они определяют модель доставки. Пакет может отправляться нескольким получателям из списка, одному или всем.

Принцип оптимальности

Прежде чем перейти к конкретным алгоритмам, сформулируем общее утверждение, описывающее оптимальные маршруты независимо от топологии или трафика, — **принцип оптимальности** (optimality principle)

Согласно этому принципу, если маршрутизатор J расположен на оптимальном пути от маршрутизатора I до маршрутизатора K, то оптимальный путь от J до K частично с ним совпадет.

Чтобы убедиться в этом, обозначим часть пути от I до J как r_1 , а остальную часть пути — r_2 . Если бы существовал более выгодный путь от J до K, чем r_2 , то его можно было объединить с r_1 , чтобы улучшить путь от I до K, что противоречит первоначальному утверждению о том, что $r_1 r_2$ — оптимальный путь.



В качестве прямого следствия принципа оптимальности мы видим, что множество оптимальных маршрутов от всех источников к заданному пункту назначения образует дерево с корнем в этом пункте назначения. Оно называется **входным деревом** (sink tree).

Расстояние в этом случае измеряется количеством транзитных участков. Цель любого алгоритма маршрутизации состоит в том, чтобы обнаружить и использовать входные деревья для всех маршрутизаторов.

Обратите внимание, что входное дерево не всегда уникально; у одной сети может быть несколько деревьев с идентичной длиной пути. Если выбрать все возможные маршруты, получится более общая структура под названием **направленный ациклический граф** (directed acyclic graph, DAG).

В таких графах нет циклов. Для удобства мы также будем называть их входным деревом. В обоих случаях предполагается, что пути не мешают друг другу (к примеру, затор на одном маршруте не приводит к изменению другого).

Входные деревья (как и любые другие) не содержат циклов: каждый пакет доставляется за конечное и ограниченное число пересылок. Но на практике все не так просто.

Маршрутизаторы (и соединения) могут отключаться и снова появляться в сети во время выполнения операции, поэтому у них может быть разное представление о текущей топологии.

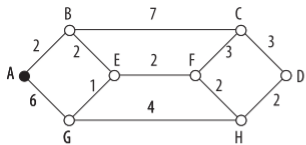
Кроме того, мы еще не обсудили, как маршрутизатор получает информацию для вычисления входного дерева. Он может самостоятельно собирать данные или получать их как-то иначе (мы рассмотрим этот вопрос чуть позже).

Алгоритм поиска кратчайшего пути

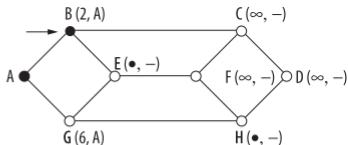
Начнем изучение алгоритмов маршрутизации с простого метода вычисления оптимальных путей с учетом полной информации о сети. Целью распределенного алгоритма является обнаружение таких путей, даже если маршрутизатор не располагает всеми сведениями о сети.

Идея заключается в построении графа сети, в котором каждый узел соответствует маршрутизатору, а каждое ребро — линии связи или ее участку. При выборе маршрута между двумя маршрутизаторами алгоритм просто находит кратчайший путь между ними на графе.

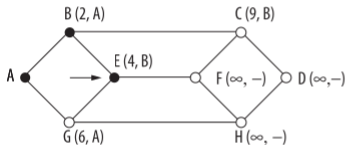
Концепция кратчайшего пути (shortes path) требует некоторого пояснения. Один из способов измерения длины пути состоит в подсчете количества транзитных участков. В этом случае пути ABC и ABE на илл. имеют одинаковую длину.



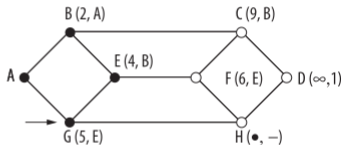
(a)



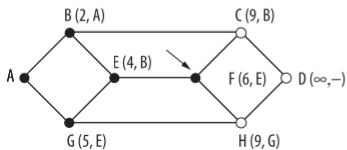
(b)



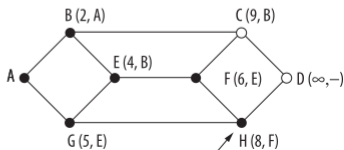
(r)



(d)



(e)



(f)

Можно измерять расстояния в километрах. Тогда получается, что ABC гораздо длиннее, чем ABE (предполагается, что рисунок изображен с соблюдением пропорций).

Однако помимо количества транзитных участков и физической длины линий есть и другие параметры. Например, каждому ребру можно присвоить метку средней задержки стандартного тестового пакета, которая измеряется каждый час. В таком графе кратчайшим является самый быстрый путь, а не путь с наименьшим количеством ребер или километров.

В общем случае метки ребер могут обозначать функции расстояния, пропускной способности, средней загруженности, стоимости связи, величины задержки и других факторов. Изменяя весовую функцию, алгоритм может вычислять «кратчайший» путь с учетом любого критерия или их комбинаций.

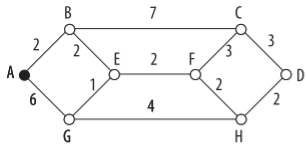
Известно несколько алгоритмов вычисления кратчайшего пути между двумя узлами графа. Один из них был создан знаменитым Дейкстрой (Dijkstra) в 1959 году. Алгоритм находит кратчайшие пути между отправителем и всеми возможными получателями в сети.

Каждому узлу присваивается метка (в скобках), означающая длину наилучшего известного пути до узла отправителя. Эти расстояния должны быть неотрицательными; условие выполняется, поскольку расстояния основаны на реальных величинах — пропускной способности или времени задержки.

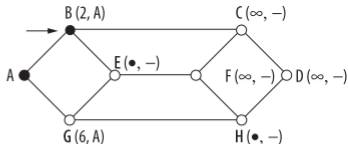
Изначально маршруты неизвестны, поэтому все узлы помечаются символом бесконечности. По мере работы алгоритма и нахождения путей метки узлов меняются, показывая оптимальные маршруты.

Метка может быть постоянной или временной. Сначала все они являются временными. Когда выясняется, что метка действительно соответствует кратчайшему пути, она становится постоянной и в дальнейшем не изменяется.

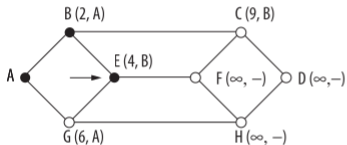
Чтобы показать, как работает этот алгоритм, рассмотрим взвешенный ненаправленный граф на илл. (а), где весовые коэффициенты отражают, например, расстояние. Нужно найти кратчайший путь от A к D . Для начала мы черным кружком помечаем узел A как постоянный. Затем исследуем все соседние с ним узлы, указывая около них расстояние до A .



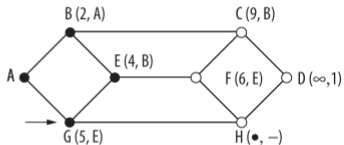
(a)



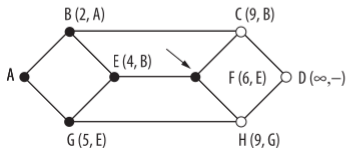
(6)



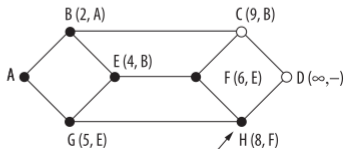
(B)



(E)



(B)



(F)

При обнаружении более короткого пути к какому-либо узлу вместе с указанием расстояния в метке меняется и узел, через который этот путь проходит. Таким образом, позже можно восстановить весь маршрут.

Если в сети несколько кратчайших путей от A до D, то, чтобы их найти, нужно запомнить все узлы, через которые можно пройти к узлу, преодолев одинаковое расстояние.

Проверив все соседние с A узлы, мы просматриваем временно помеченные узлы во всем графе и делаем узел с наименьшей меткой постоянным (илл. (6)). Он становится новым рабочим узлом.

Теперь мы повторяем ту же процедуру с узлом В, исследуя все ближайшие узлы. Сначала складываем расстояние от В до соседнего узла и значение метки В (то есть расстояние от А до В). Если полученная сумма меньше, чем метка исследуемого узла (то есть расстояние от А, уже найденное на предыдущем этапе), значит, обнаружен более короткий путь, поэтому метка узла меняется.

После того как все узлы, примыкающие к рабочему, исследованы и временные метки изменены (если это возможно), по всему графу ищется узел с наименьшей временной меткой. Он помечается как постоянный и становится текущим рабочим узлом.

Чтобы понять работу алгоритма, посмотрим на илл. 5.7 (в). На данном этапе узел E только что был отмечен как постоянный. Предположим, что существует более короткий путь, нежели ABE, например AXYZE (для некоторых X и Y). Существует две возможности — либо узел Z уже постоянный, либо еще нет. Если да, значит, узел E уже проверялся, когда узел Z был сделан постоянным и, следовательно, рабочим узлом. В этом случае путь AXYZE уже исследовался.

Теперь рассмотрим ситуацию, когда узел Z все еще помечен как временный. Если метка узла Z больше или равна метке узла E , путь $AXYZE$ не может быть короче, чем путь ABE . Если же метка узла Z меньше метки узла E , тогда узел Z стал бы постоянным раньше узла E и узел E проверялся бы с узла Z

```

#define MAX_NODES 1024 /* максимальное количество узлов */
#define INFINITY 100000000 /* число, превышающее длину максимального пути */
int n, dist[MAX_NODES][MAX_NODES]; /* dist[i][j] – это расстояние от i до j */

void shortest_path(int s, int t, int path[])
{
    struct state { /* рабочий путь */
        int predecessor; /* предыдущий узел */
        int length; /* расстояние от источника до этого узла */
        enum {permanent, tentative} label; /* метка состояния */
    } state[MAX_NODES];

    int i, k, min;
    struct state *p;

    for (p = &state[0]; p < &state[n]; p++) { /* инициализация состояний */
        p->predecessor = -1;
        p->length = INFINITY;
        p->label = tentative;
    }
    state[t].length = 0; state[t].label = permanent;
    k = t; /* k – начальный рабочий узел */
    do { /* Есть ли лучший путь от k? */
        /* У этого графа n узлов */
        for (i = 0; i < n; i++)
            if (dist[k][i] != 0 && state[i].label == tentative) {
                if (state[k].length + dist[k][i] < state[i].length) {
                    state[i].predecessor = k;
                    state[i].length = state[k].length + dist[k][i];
                }
            }

        /* Поиск узла, предварительно помеченного наименьшей меткой */
        k = 0; min = INFINITY;
        for (i = 0; i < n; i++)
            if (state[i].label == tentative && state[i].length < min) {
                min = state[i].length;
                k = i;
            }
        state[k].label = permanent;
    } while (k != s);

    /* Копирование пути в выходной массив */
    i = 0; k = s;
    do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}

```


Пример реализации алгоритма представлен на илл. Глобальные переменные `n` и `dist` описывают граф и инициализируются до вызова функции `shortest_path`. Единственное отличие программы от описанного выше алгоритма заключается в том, что вычисление кратчайшего пути в программе начинается не с узла-источника s , а с конечного узла t .

Поскольку в однонаправленном графе кратчайшие пути от t к s те же, что и от s к t , не имеет значения, с какого конца начинать. Причина поиска пути в обратном направлении заключается в том, что каждый узел помечается предшествующим узлом, а не следующим. Когда найденный маршрут копируется в выходную переменную `path`, он инвертируется. В результате двух инверсий получается путь в нужном направлении.

При выполнении алгоритма маршрутизации любой маршрутизатор должен принимать решения на основании локальных сведений, а не полной информации о сети.

Одним из простых локальных методов является **лавинная адресация** (flooding), при которой каждый входящий пакет отправляется на все исходящие линии, кроме той, по которой он пришел.

Очевидно, что этот алгоритм порождает огромное, даже бесконечное количество дублированных пакетов, если не принять меры.

Например, можно поместить в заголовок пакета счетчик транзитных участков, который уменьшается при прохождении каждого маршрутизатора. Когда значение этого счетчика падает до нуля, пакет удаляется.

В идеале счетчик устанавливается согласно длине пути от источника до адресата. Если отправитель не знает это расстояние, он может установить значение счетчика в соответствии с наихудшим случаем, то есть диаметром сети.

